

Chapter 11. Personalized Recommendation Metrics

Having explored the powerful methodologies of MF and neural networks in the context of personalization, we are now equipped with potent tools to craft sophisticated recommendation systems. However, the order of recommendations in a list may have a profound impact on user engagement and satisfaction.

Our journey so far has primarily been focused on predicting what a user may like, using latent factors or deep learning architectures. However, the manner in which we present these predictions, or more formally, how we rank these recommendations, holds paramount significance. Therefore, this chapter will shift our gaze from the prediction problem and will unravel the complex landscape of ranking in recommendation systems.

This chapter is dedicated to understanding key ranking metrics including mean average precision (mAP), mean reciprocal rank (MRR), and normalized discounted cumulative gain (NDCG). Each of these metrics takes a unique approach toward quantifying the quality of our rankings, catering to different aspects of the user interaction.

We'll dive into the intricacies of these metrics, unveiling their computational details and discussing their interpretation, covering their strengths and weaknesses, and pointing out their specific relevance to various personalization scenarios.

This exploration forms an integral part of the evaluation process in recommendation systems. It not only gives us a robust framework to measure the performance of our system but also provides essential insights into understanding how different algorithms might perform in online set-

tings. This will lay the foundation for future discussions on algorithmic bias, diversity in recommendations, and a multistakeholder approach to recommendation systems.

In essence, the knowledge garnered in this chapter will be instrumental in fine-tuning our recommendation system, ensuring that we don't just predict well but also recommend in a way that truly resonates with individual user preferences and behaviors.

Environments

Before we dig into defining the key metrics, we're going to spend a few moments discussing the kinds of evaluation we can do. Evaluation for recommendation systems, as you'll soon see, is frequently characterized by how *relevant* the recommendations are for a user. This is similar to search metrics, but we add in additional factors to account for *where* in the list the most relevant items are.

For an extremely comprehensive view on evaluation of recommender systems, the recent project [RecList](#) builds a useful checklist-based framework for organizing metrics and evaluations.

Often you'll hear about evaluating recommenders in a few setups:

- Online/offline
- User/item
- A/B

Each setup provides slightly different kinds of evaluations and tells you different things. Let's quickly break down the differences to set some assumptions about terminology.

Online and Offline

When we refer to online versus offline recommenders, we are referring to *when* you're running evals. In *offline evaluation*, you start with a test/evaluation dataset, outside your production system, and compute a

set of metrics. This is often the simplest recommender to set up but has the highest expectation of existing data. Using historical data, you construct a set of relevant responses, which you can then use during simulated inference. This approach is the most similar to other kinds of traditional ML, although with slightly different computations for the error.

When we're training large models, these datasets are similar to an offline dataset. We previously saw prequential data, which is much more relevant in recommendation systems than in lots of other ML applications. Sometimes you'll hear people say that "all recommenders are sequential recommenders" because of the importance of historical exposure to the recommender problem.

Online evaluation takes place during inference, usually in production. The tricky part is that you essentially never know the counterfactual outcomes. You can compute specific metrics on the online rankings: frequency and distributions of covariates, CTR/success rate, or time on platform, but ultimately these are different from the offline metrics.

BOOTSTRAPPING FROM HISTORICAL EVALUATION DATA

One of the most common questions from people building a recommender from scratch is "Where do you get the initial training data?" This is a hard problem. Ultimately, you have to be clever to come up with a useful dataset. Consider our co-occurrence data in the Wikipedia recommender; we didn't require any user interactions to get to a set of data to build a recommender. Bootstrapping from item to item is the most popular strategy, but you can use other tricks as well. The simplest way to start moving into user-item recommenders is to simply ask the user questions. If you ask for preference information across a set of item features, you can build simple models that start to incorporate this.

User Versus Item Metrics

Because recommender systems are personalization machines, it can be easy to think that we always want to be making recommendations for the user and measuring the performance as such. Subtleties exist, though. We want to be sure individual items are getting a fair chance, and sometimes

looking at the other side of the equation can help assess this. In other words, are the items getting recommended frequently enough to have a chance to find their niche? We should explicitly compute our metrics over user *and* item axes.

Another aspect of item-side metrics is for set-based recommenders. The other items that are recommended in context can have a significant effect on the performance of a recommendation. As a result, we should be careful to measure the pairwise item metrics in our large-scale evaluations.

A/B Testing

It's good to use randomized, controlled trials to evaluate how your new recommendation model is performing. For recommendations, this is quite tricky. At the end of this chapter, you'll see some of the nuance, but for now, let's consider a quick reminder of how to think about A/B testing in a closed-loop paradigm.

A/B tests ultimately attempt to estimate the effect size of swapping one model in for another; effect size estimation is the process of measuring the causal impact of an intervention on a target metric. First, we would need to deploy two recommender models. We'd also hope that there's a reasonable randomization of users into each of the recommenders. However, what's the randomization unit? It's easy to quickly assume it's the user, but what has changed about the recommender? Has the recommender changed in a way that covaries with some properties of the distribution—e.g., have you built a new recommender that is less friendly toward seasonal TV specials just as we enter into the second week of November?

Another consideration with this sort of testing for recommendation systems is the long-term compounding effects. A frequent rejoinder about a series of positive A/B test outcomes over several years is “Have you tested the first recommender against the last?” This is because populations change, both the users and the items. As you also vary the recommender system, you frequently find yourself in a double-blind situation where you've never seen this user or item population with any other recom-

mender. If all the effect sizes of every A/B test were additive across the industry, the world GDP would likely be two to three times as large.

The way to guard against protests like this is via a *long-term holdout*, a random subset of users (continually being added to) who will not be upgraded to new models through time. By measuring the target metrics on this set versus the most cutting-edge model in production, you're always able to understand the long-term effects of your work. The downside of a long-term holdout? It's hard to maintain, and it's hard to sacrifice some of the effects of your work on a subset of the population.

Now let's finally get to the metrics already!

Recall and Precision

Let's begin by considering four recommender problems and how each may have different implications for the kind of results you want.

First, let's consider entering a bookstore and looking for a book by a popular author. We would say this is the recommender problem:

- Provides a lot of recommendations
- Offers few possible relevant results

Additionally, if the bookstore has a good selection, we'd expect that *all* the relevant results are contained in the recommendations because bookstores often carry most or all of an author's oeuvre once they've become popular. However, many of the recommendations—the books in the bookstore—are simply not relevant for this search.

Second, let's consider looking for a gas station nearby on a mapping app while in a large metro. We expect that a lot of gas stations are relatively close by, but you would probably consider only the first couple—or maybe even only one, the first one that you see. Thus a recommender for this problem has the following:

- Many relevant results

- Few useful recommendations

In the first scenario, the relevant results may be fully contained in the recommendations, and in the second scenario, the recommendations may be fully contained in the relevant results.

Let's now look at more common scenarios.

For our third example, consider that you're searching on a streaming video platform for something to watch tonight when you're feeling romantic. Streaming platforms tend to show a lot of recommendations—pages and pages from this one theme or another. But on this night, and on just this platform, only a couple of movies or TV shows might really fit what you're looking for. Our recommender, then, does the following:

- Provides many recommendations
- Offers only a few that are actually relevant

However, importantly, not all relevant results will be in the recommendations! As we know, different platforms have different media, so some of the relevant results won't appear in the recommendations no matter how many we look at.

Fourth, and finally, you're a high-end coffee lover with distinguished tastes headed into the local roaster for a third-wave, single-origin coffee. As an experienced coffee connoisseur, you love high-quality coffees from all over the world and enjoy most but not all origins. On any given day, your local cafe has only a few single-origin hand-brewed options. Despite your worldly palette, there are some popular terroirs that you just don't love. This little recommendation brew bar can be described as follows:

- Provides a few recommendations
- Offers many possible recommendations that are relevant

On any given day, only some of the few recommendations may be relevant to you.

So those are our matching four scenarios. For the latter two, the intersection between recommendation and relevance may be proportionally small or large—or even empty! The main idea is that the full size of the smaller sample is not always in use.

Now that we’ve worked through a few examples, let’s see how they relate to the core metrics for a recommender: precision and recall @ k ([Figure 11-1](#)). Focusing on examples 3 and 4, we can see that only some of the recommendations intersect with the options that are relevant. And only some of the relevant options intersect with the recommendations. It’s often overlooked, but in fact *these two ratios define our metrics*—let’s go!

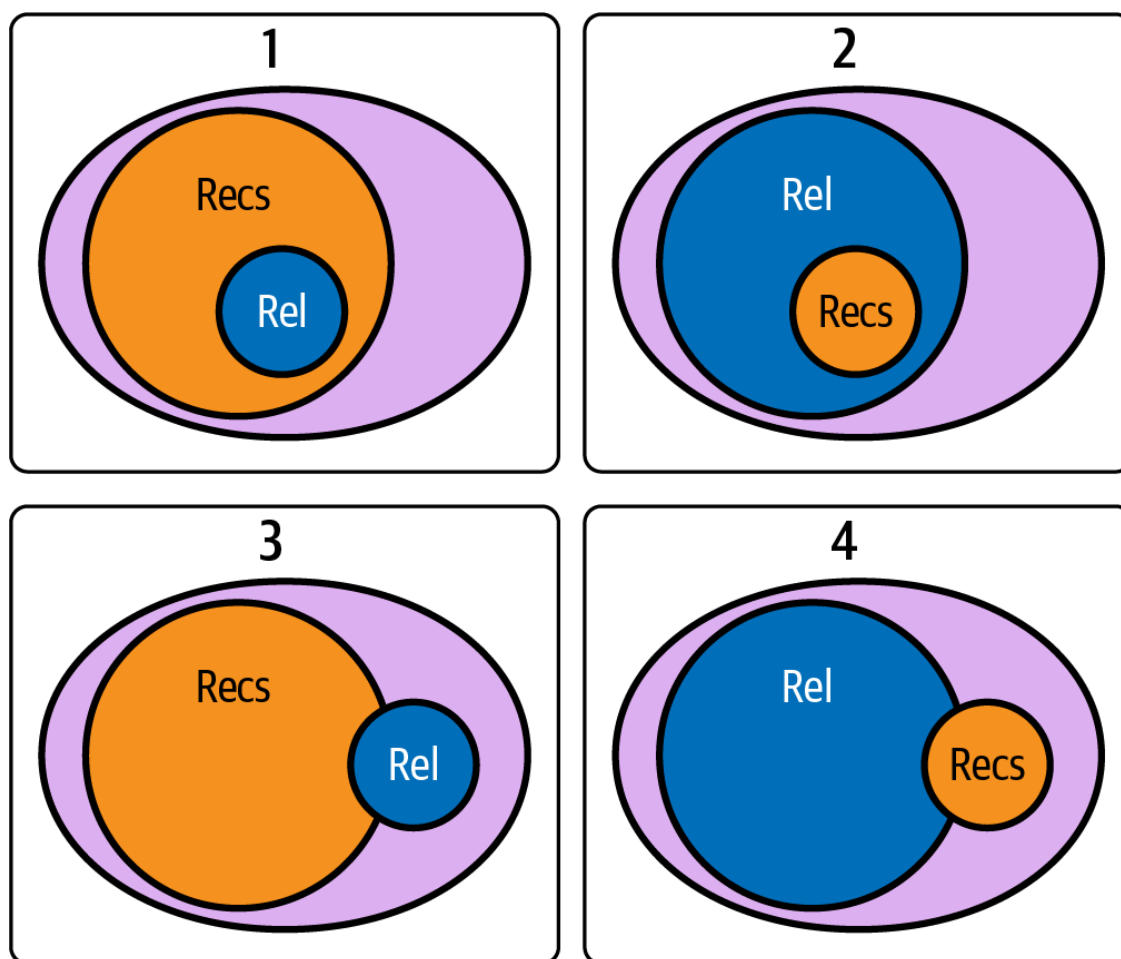


Figure 11-1. Recall and precision sets

@ k

In much of this chapter and RecSys metrics discussion, we say things like @ k . This means “at k ,” which should really be “in k ” or “out of k .” These are simply the size of the set of recommendations. We often anchor the

customer experience on how many recommendations we can show the user without the experience suffering. We also need to know the cardinality of the set of relevant items, which we call $@ r$. Note that while it may not feel like it's possible to ever know this number, we assume this refers to “known relevant” options via our training or test data.

Precision at k

Precision is the ratio of the size of the set of relevant recommendations to k , the size of the set of recommendations.

$$Precision@k = \frac{num_{relevant}}{(k)}$$

Notice that the size of the relevant items doesn't appear in the formula. That's OK; the size of the intersection is still dependent on the size of the set of relevant items.

Looking at our examples, 2 technically has the highest precision, but it's a bit of a red herring because of the number of relevant results. This is one reason precision is not the most common metric for evaluating recommendation systems.

Recall at k

Recall is the ratio of the size of the set of relevant recommendations to r , the size of the set of relevant items.

But wait! If the ratio is the relevant recommendations over the relevant items, where is k ? k is still important here because the size of the set of recommendations constrains the possible size of the intersection. Recall that these ratios are operating on that intersection that is always dependent on k . This means you often consider the max of r and k .

In scenario 3, we hope that some of the movies that fit our heart's desire will be on the right streaming platform. The number of these divided by the count of all the media anywhere is the *recall*. If all your relevant movies are on this platform, you might call that *total recall*.

Scenario 4’s café experience shows that recall is sometimes the inverse probability of an avoid; because you like so many coffees, we might instead find it easier to talk about what you don’t like. In this case, the number of avoids in the offering will have a large effect on the recall:

$$Recall@k = \frac{(k - Avoid@k)}{num_{relevant}}$$

This is the core mathematical definition for recall and is often one of the first measurements we’ll consider because it’s a pure estimate of how your retrieval is performing.

R-precision

If we also have a ranking on our recommendations, we can take the ratio of relevant recommendations to r in the *top- r* recommendations. This improves this metric in cases where r is very small, as in examples 1 and 3.

mAP, MMR, NDCG

Having delved into the reliable domains of $precision@k$ and $recall@k$, we’ve gained valuable insights into the quality of our recommendation systems. However, these metrics, as crucial as they are, can sometimes fall short in capturing an important aspect of these systems: *the order of recommendations*.

In recommendation systems, the ordering in which we present suggestions carries significant weight and needs to be evaluated to ensure that it’s effective.

That’s why we’ll now journey beyond $precision@k$ and $recall@k$ to explore some key ranking-sensitive metrics—namely, mean average precision (mAP), mean reciprocal rank (MRR), and normalized discounted cumulative gain (NDCG). These metrics consider not only whether our recommendations are relevant but also whether they are well-ordered.

The mAP metric lends importance to each relevant document and its position, and MRR concentrates on the rank of the first relevant item. NDCG gives more importance to relevant documents at higher ranks. By understanding these metrics, you'll have an even more robust set of tools to evaluate and refine your recommendation systems.

So, let's carry on with our exploration, striking a balance between precision and comprehensibility. By the end of this section, you will be well equipped to handle these essential evaluation methods in a confident and knowledgeable manner.

mAP

This vital metric in recommendation systems is particularly adept at accounting for the rank of relevant items. If, in a list of five items, the relevant ones are found at positions 2, 3, and 5, mAP would be calculated by computing precision@2, precision@3, and precision@5 and then taking an average of these values. The strength of mAP lies in its sensitivity to the ordering of relevant items, providing a higher score when these items are ranked higher.

Consider an example with two recommendation algorithms A and B:

- For algorithm A, we compute the mAP as follows:
 $(\text{precision@2} + \text{precision@3} + \text{precision@5}) / 3 = (1/2 + 2/3 + 3/5) / 3 = 0.6$
- For algorithm B, which perfectly ranks the items, we calculate mAP as follows:
 $\text{mAP} = (\text{precision@1} + \text{precision@2} + \text{precision@3}) / 3 = (1/1 + 2/2 + 3/3) / 3 = 1$

The generalized formula for mAP across a set of queries Q is shown here:

$$mAP = \frac{1}{|Q|} \sum_{q=1}^{|Q|} \frac{1}{m_q} \sum_{k=1}^n P(k) * rel(k)$$

Here, $|Q|$ is the total number of queries, m_q is the number of relevant documents for a specific query q , $P(k)$ stands for the precision at the

$_k$ _th cutoff, and $rel(k)$ is an indicator function equating to 1 if the item at rank k is relevant, and 0 otherwise.

MRR

Another effective metric used in recommendation systems is MRR. Unlike MAP, which considers all relevant items, MRR primarily focuses on the position of the first relevant item in the recommendation list. It's computed as the reciprocal of the rank where the first relevant item appears.

Consequently, MRR can reach its maximum value of 1 if the first item in the list is relevant. If the first relevant item is found farther down the list, MRR takes a value less than 1. For instance, if the first relevant item is positioned at rank 2, the MRR would be $1/2$.

Let's look at this in the context of the recommendation algorithms A and B that we used earlier:

- For algorithm A, the first relevant item is at rank 2, so the MRR equals $1/2 = 0.5$.
- For algorithm B, which perfectly ranked the items, the first relevant item is at rank 1, so the MRR equals $1/1 = 1$.

Extending this to multiple queries, the general formula for MRR is as follows:

$$MRR = \frac{1}{|Q|} \sum_{i=1}^{|Q|} \frac{1}{rank_i}$$

Here, $|Q|$ represents the total number of queries, and $rank_i$ is the position of the first relevant item in the list for the $_i$ _th query. This metric provides valuable insight into how well a recommendation algorithm delivers a relevant recommendation right at the top of the list.

NDCG

To further refine our understanding of ranking metrics, let's step into the world of NDCG. Like mAP and MRR, NDCG also acknowledges the rank order of relevant items but introduces a twist. It discounts the relevance of

items as we move down the list, signifying that items appearing earlier in the list are more valuable than those ranked lower.

NDCG begins with the concept of cumulative gain (CG), which is simply the sum of the relevance scores of the top k items in the list. Discounted cumulative gain (DCG) goes a step further, discounting the relevance of each item based on its position. NDCG, then, is the DCG value normalized by the ideal DCG (IDCG), the DCG that we would obtain if all relevant items appeared at the very top of the list.

Assuming we have five items in our list and a specific user for whom the relevant items are found at positions 2 and 3, the $IDCG@k$ would be $(1/\log(1 + 1) + 1/\log(2 + 1)) = 1.5 + 0.63 = 2.13$.

Let's put this into the context of our example algorithms A and B.

For algorithm A

- $DCG@5 = 1/\log(2 + 1) + 1/\log(3 + 1) + 1/\log(5 + 1) = 0.63 + 0.5 + 0.39 = 1.52$
- $NDCG@5 = DCG@5 / IDCG@5 = 1.52 / 2.13 = 0.71$

For algorithm B

- $DCG@5 = 1/\log(1 + 1) + 1/\log(2 + 1) + 1/\log(3 + 1) = 1 + 0.63 + 0.5 = 2.13$
- $NDCG@5 = DCG@5 / IDCG@5 = 2.13 / 2.13 = 1$

The general formula for NDCG can be represented as

$$NDCG@k = \frac{DCG@k}{IDCG@k}$$

where

- $DCG@k = \sum_{i=1}^k \frac{rel_i}{\log_2(i+1)}$
- $IDCG@k = \sum_{i=1}^{|\mathcal{R}|} \frac{1}{\log_2(i+1)}$

and $\mathcal{R}$ is the set of relevant documents.

This metric gives us a normalized score for how well our recommendation algorithm ranks relevant items, discounting as we move further down the list.

mAP Versus NDCG?

Both mAP and NDCG are holistic metrics that offer a comprehensive perspective of ranking quality by incorporating all relevant items and their respective ranks. However, the interpretability and use cases of these metrics can vary based on the specifics of the recommendation context and the nature of relevance.

While MRR does not consider all relevant items, it does provide an interpretable insight into an algorithm's performance, highlighting the average rank of the first relevant item. This can be particularly useful when the topmost recommendations hold significant value.

mAP, on the other hand, is a rich evaluation measure that effectively represents the area under the precision-recall curve. Its average aspect confers an intuitive interpretation related to the trade-off between precision and recall across different rank cutoffs.

NDCG introduces a robust consideration of the relevance of each item and is sensitive to the rank order, employing a logarithmic discount factor to quantify the diminishing significance of items as we move down the list. This allows it to handle scenarios in which items can have varying degrees of relevance, extending beyond binary relevance often used in mAP and MRR. However, this versatility of NDCG can also limit its interpretability because of the complexity of the logarithmic discount.

Further, although NDCG is well equipped for use cases where items carry distinct relevance weights, procuring accurate ground-truth relevance scores can pose a significant challenge in practical applications. This imposes a limitation on the real-world usefulness of NDCG.

Cumulatively, these metrics form the backbone of offline evaluation methodologies for recommendation algorithms. As we advance in our ex-

ploration, we'll cover online evaluations, discuss strategies to assess and mitigate algorithmic bias, understand the importance of ensuring diversity in recommendations, and optimize recommendation systems to cater to various stakeholders in the ecosystem.

Correlation Coefficients

While correlation coefficients like Pearson's or Spearman's can be employed to evaluate the similarity between two rankings (for instance, between the predicted and the ground-truth rankings), they do not provide the exact same information as mAP, MRR, or NDCG.

Correlation coefficients are typically used to measure the degree of linear association between two continuous variables, and in the context of ranking, they can indicate the overall similarity between two ordered lists. However, they do not directly account for aspects such as the relevance of individual items, the position of relevant items, or varying degrees of relevance among items, which are integral to mAP, MRR, and NDCG.

For example, say a user has interacted with five items in the past. A recommender system might predict that the user will interact with these items again but rank them in the opposite order of importance. Even though the system has correctly identified the items of interest, the reversed ranking would lead to poor performance as measured by mAP, MRR, or NDCG, but a high negative correlation coefficient would be obtained because of the linear relationship.

As a result, while correlation coefficients can provide a high-level understanding of ranking performance, they are not sufficient substitutes for the more detailed information provided by metrics like mAP, MRR, and NDCG.

To utilize correlation coefficients in the context of ranking, it would be essential to pair them with other metrics that account for the specific nuances of the recommendation problem, such as the relevance of individual items and their positions in the ranking.

RMSE from Affinity

Root mean square error (RMSE) and ranking metrics like mAP, MRR, and NDCG offer fundamentally different perspectives when evaluating a recommendation system that outputs affinity scores.

RMSE is a popular metric for quantifying prediction error. It computes the square root of the average of squared differences between the predicted affinity scores and the true values. Lower RMSE signifies better predictive accuracy. However, RMSE treats the problem as a standard regression task and disregards the inherent ranking structure in recommendation systems.

Conversely, mAP, MRR, and NDCG are explicitly designed to evaluate the quality of rankings, which is essential in a recommendation system. In essence, while RMSE measures the closeness of predicted affinity scores to actual values, mAP, MRR, and NDCG assess the ranking quality by considering the positions of relevant items. Therefore, if your main concern is ranking items rather than predicting precise affinity scores, these ranking metrics are generally more appropriate.

Integral Forms: AUC and cAUC

When it comes to recommendation systems, we are producing a ranked list of items for each user. As you've seen, these rankings are based on affinity, the probability or level of preference that the user has for each item. Given this framework, several metrics have been developed to evaluate the quality of these ranked lists. One such metric is the AUC-ROC, which is complemented by mAP, MRR, and NDCG. Let's take a closer look at understanding these.

Recommendation Probabilities to AUC-ROC

In a binary classification setup, the area *under the receiver operating characteristic curve* (AUC-ROC) measures the ability of the recommendation model to distinguish between positive (relevant) and negative (irrelevant)

instances. It is calculated by plotting the true positive rate (TPR) against the false positive rate (FPR) at various threshold settings and then computing the area under this curve.

In the context of recommendations, you can think of these “thresholds” as varying the number of top items recommended to a user. The AUC-ROC metric becomes an evaluation of how well your model ranks relevant items over irrelevant ones, irrespective of the actual rank position. In other words, AUC-ROC effectively quantifies the likelihood that a randomly chosen relevant item is ranked higher than a randomly chosen irrelevant one by the model. This, however, doesn’t account for the actual position or order of items in the list, only the relative ranking of positive versus negative instances. The affinity of a calibrated item may be interpreted as a confidence measure by the model that an item is relevant, and when considering historical data, even uncalibrated affinity scores may make a great suggestion for the number of recommendations necessary to find something useful.

One serious implementation of these affinity scores might be to show users only items over a particular score and otherwise tell them to come back later or use exploration methods to improve the data. For example, if you sold hygiene products and were considering asking customers to add some Aesop soap during checkout, you may wish to evaluate the Aesop ROC and make this suggestion only when the observed affinity passed the learned threshold. You’ll also see these concepts used later in [“Inventory Health”](#).

Comparison to Other Metrics

Let’s put these in context with the other metrics:

mAP

This metric expands on the idea of precision at a specific cutoff in the ranked list to provide an overall measure of model performance. It does this by averaging the precision values computed at the ranks where each relevant item is found. Unlike AUC-ROC, mAP

puts emphasis on the higher-ranked items and is more sensitive to changes at the top of the ranking.

MRR

Unlike AUC-ROC and mAP, which consider all relevant items in the list, MRR focuses only on the rank of the first relevant item in the list. It is a measure of how quickly the model can find a relevant item. If the model consistently places a relevant item at the top of the list, it will have a higher MRR.

NDCG

This metric evaluates the quality of the ranking by not only considering the order of recommendations but also taking into account the graded relevance of items (which the previous metrics don't). NDCG discounts items further down the list, rewarding relevant items that appear near the top of the list.

AUC-ROC provides a valuable aggregate measure of a model's ability to differentiate between relevant and irrelevant items; mAP, MRR, and NDCG offer a more nuanced evaluation of the model's ranking quality, considering factors like position bias and varying degrees of relevance.

Note that we sometimes compute the AUC per customer and then average. That's customer AUC (cAUC), which can often provide a good expectation for a user's experience.

BPR

Bayesian personalized ranking (BPR) presents a Bayesian approach to the task of item ranking in recommendation systems, effectively providing a probability framework to model the personalized ranking process.

Instead of transforming the item recommendation problem into a binary classification problem (relevant or not), BPR focuses on pairwise preferences: given two items, which does the user prefer? This approach aligns better with the nature of implicit feedback that is common in recommendation systems.

The BPR model uses a pairwise loss function that takes into account the relative order of a positive item and a negative item for a specific user. It seeks to maximize the posterior probability of the observed rankings being correct. The model is typically optimized using stochastic gradient descent or a variant thereof. It's important to note that BPR (unlike other metrics we've discussed, including AUC-ROC, mAP, MRR, and NDCG) is a model training objective rather than an evaluation metric. Therefore, while the aforementioned metrics evaluate a model's performance post-training, BPR provides a mechanism to guide the model learning process in a way that directly optimizes for the ranking task. A much deeper discussion of these topics is in [“BPR: Bayesian Personalized Ranking from Implicit Feedback”](#) by Steffen Rendle et al.

Summary

Now that you know how to evaluate the performance of the recommendation systems that you train, you may be wondering how to actually train them. You may have noticed that many of the metrics we introduced would not make very good loss functions; they involve a lot of simultaneous observations about sets and lists of items. This would unfortunately make the signal that the recommender would be learning from highly combinatorial. Additionally, the metrics we've presented really have two aspects to consider: the binary metric associated to recall, and the rank weighting.

In the next chapter, you're going to learn some loss functions that make excellent training objectives. The importance of these, we're sure, won't be lost on you.